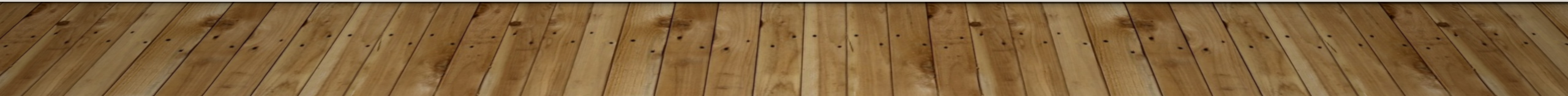


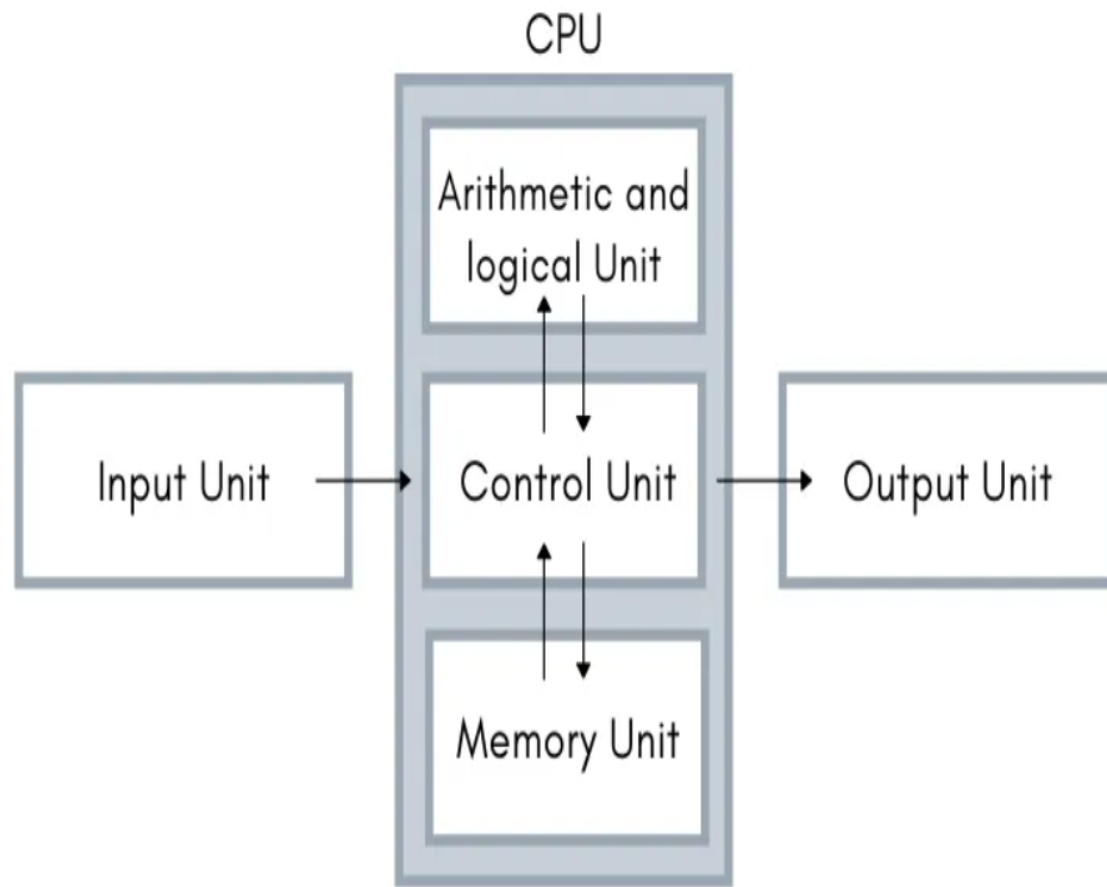
FUNDAMENTAL CONCEPTS

UNIT 2

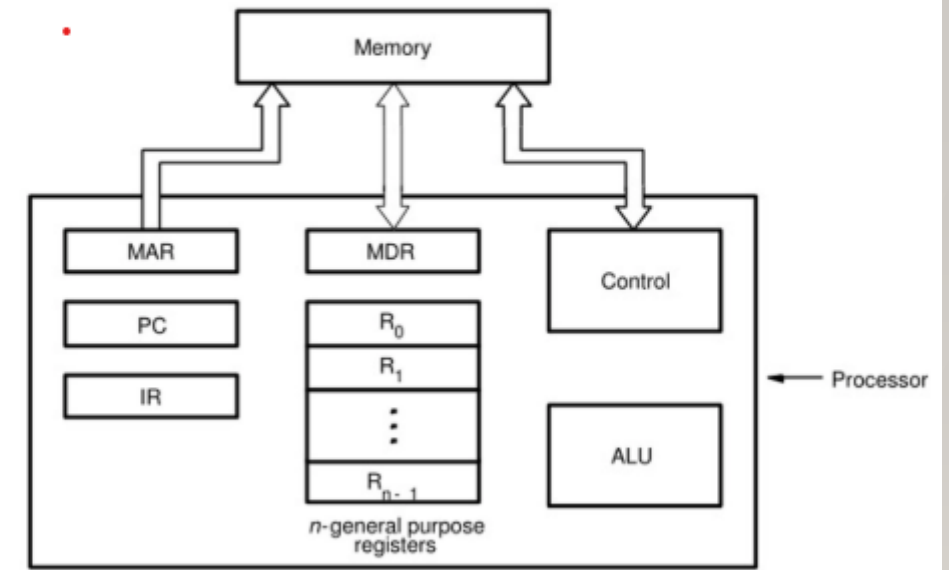


Fundamental Concepts

- Processor fetches one instruction at a time and perform the operation specified.
- Instructions are fetched from successive memory locations until a branch or a jump instruction is encountered.
- Processor keeps track of the address of the memory location containing the next instruction to be fetched using Program Counter (PC).
- Instruction Register (IR)



Connection Between the Processor and the Memory



Connections between the processor and the memory.

Register

The registers are an integral part of the CPU. They are a type of **memory** that can be accessed very quickly compared to other types of memory. The pieces of information they hold are needed very often. They can be used to **store data** and **control information** during a fetch-decode-execute cycle or they can be used to hold values that are generated as part of the ALU working on data.

SOME OF THE REGISTERS

- **Program Counter (PC)**
- **Instruction Register (IR)**
- **Memory Address Register (MAR)**
- **Memory Data Register (MDR)**
- **General Purpose Registers**

❖ General Purpose Registers

A register is a collection of flip-flops. Single bit digital data is stored using flip-flops. By combining many flip-flops, the storage capacity can be extended to accommodate a huge number of bits. We must utilize an n -bit register with n flip flops if we wish to store an n -bit word.

General Purpose Registers (GPRs) are essential components within a CPU, serving as temporary storage locations for data that is actively being processed. Unlike special-purpose registers, which are dedicated to specific tasks, GPRs are versatile and can store a wide variety of data, including operands for arithmetic and logic operations, memory addresses, or intermediate results.

They provide fast access to frequently used data, reducing the need to fetch from slower main memory and thereby speeding up program execution. Their contents can be read/written by programmer /user through instructions.

❖ PROGRAM COUNTER

The program counter, also known as the **instruction pointer** or simply PC, is a fundamental component of a computer's central processing unit (CPU). It is a special register that keeps track of the memory address of the next instruction to be executed in a program.

❖ INSTRUCTION REGISTER

The **Instruction Register (IR)** is a crucial component of a computer's Central Processing Unit (CPU). It holds the instruction currently being executed or decoded. The IR is part of the control unit of the CPU and plays a vital role in the instruction cycle, which includes fetching, decoding, and executing instructions.

Function and Workflow

The instruction register temporarily stores the instruction fetched from memory. This instruction is then decoded to determine the operation to be performed and the operands involved. The decoded instruction is executed by the CPU, and the results are stored back in memory or other registers.

Step-by-Step Process

1. **Fetch Phase:** The CPU fetches the next instruction from memory and loads it into the instruction register. This step ensures that the CPU knows what operation to execute next.
2. **Decode Phase:** The instruction in the IR is decoded to interpret the command. This involves determining the operation and the operands.
3. **Execute Phase:** The decoded instruction is executed. For example, an addition operation would add the contents of two registers and store the result in a third register.

❖ **Memory Address Register (MAR)**

The **Memory Address Register (MAR)** is a crucial component within a computer's CPU. It is responsible for storing the memory address from which data will be fetched to the CPU registers or the address to which data will be sent and stored via the system bus. Essentially, the MAR holds the memory location of data that needs to be accessed during the execution phase of an instruction.

KEY FUNCTIONS OF MAR

- **Fetching Data:** When the CPU needs to read data from memory, the MAR holds the address of the data to be fetched.
- **Storing Data:** When the CPU needs to write data to memory, the MAR holds the address where the data will be stored.
- **Coordination with MDR:** The MAR works in conjunction with the Memory Data Register (MDR) to facilitate the transfer of data between the CPU and memory.

❖ Memory Data Register (MDR)

The Memory Data Register (MDR) The Memory Data Register (MDR) is a **crucial component in CPU architecture**, serving as a buffer for data during transfer to and from main memory. It works with the Memory Address Register (MAR) to facilitate read and write operations, influencing the speed and efficiency of data processing.

Executing an Instruction

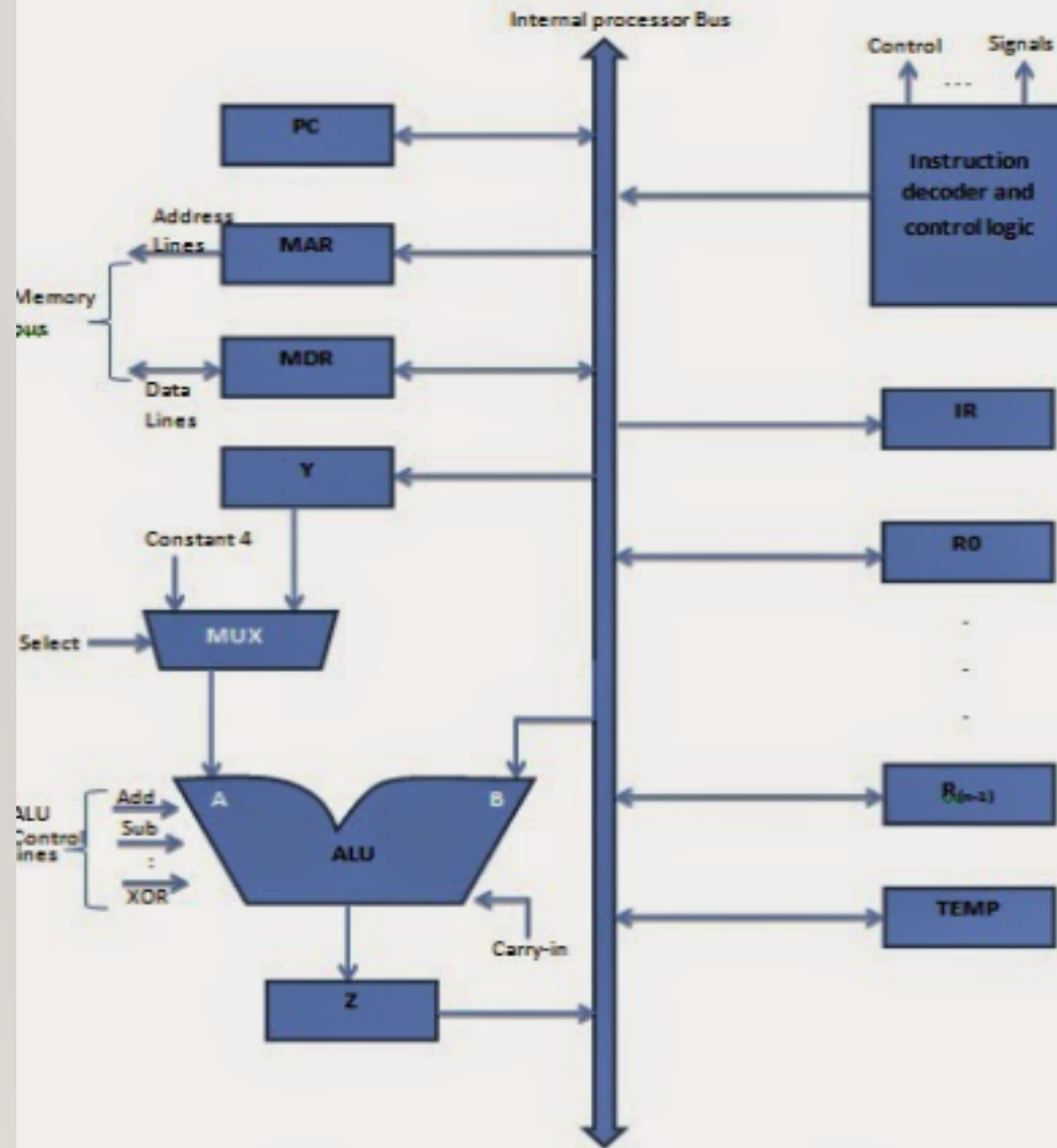
- Fetch the contents of the memory location pointed to by the PC. The contents of this location are loaded into the IR (fetch phase).

$$IR \leftarrow [[PC]]$$

- Assuming that the memory is byte addressable, increment the contents of the PC by 4 (**Fetch phase**).

$$PC \leftarrow [PC] + 4$$

- Carry out the actions specified by the instruction in the IR (**Execution phase**).



SINGLE BUS ORGANISATION

Single Bus Organization

- ALU
- Registers for temporary storage
- Various digital circuits for executing different micro operations.(gates, MUX, decoders, counters).
- Internal path for movement of data between ALU and registers.
- Driver circuits for transmitting signals to external units.
- Receiver circuits for incoming signals from external units.

Single Bus Organization - contd.

Program Counter (PC)

- Keeps track of execution of a program
- Contains the memory address of the next instruction to be fetched and executed.

Memory Address Register (MAR)

- Holds the address of the location to be accessed.
- I/P of MAR is connected to Internal bus and an O/P to external bus.

Memory Data Register (MDR)

- It contains data to be written into or read out of the addressed location.
- It has 2 inputs and 2 outputs.
- Data can be loaded into MDR either from memory bus or from internal processor bus.
- The data and address lines are connected to the internal bus via MDR and MAR

Single Bus Organization - contd.

Registers

- The processor registers R_0 to R_{n-1} vary considerably from one processor to another.
- Registers are provided for general purpose used by programmer.
- Special purpose registers-index & stack registers.
- Registers Y, Z & TEMP are temporary registers used by processor during the execution of some instruction.

Multiplexer

- Select either the output of the register Y or a constant value 4 to be provided as input A of the ALU.
- Constant 4 is used by the processor to increment the contents of PC.

Single Bus Organization - contd.

ALU

- B input of ALU is obtained directly from processor-bus.
- As instruction execution progresses, data are transferred from one register to another, often passing through ALU to perform arithmetic or logic operation.

Data Path

- The registers, ALU and interconnecting bus are collectively referred to as the data path.

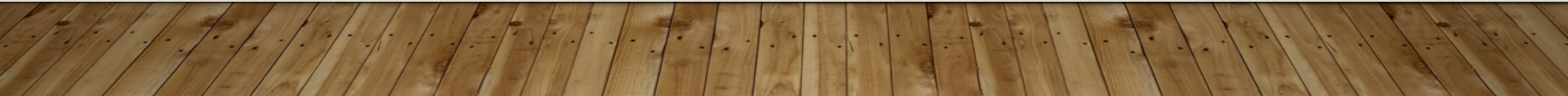
❖ DATA PATH

A **data path** (or datapath) is a critical component of a computer's architecture, consisting of various functional units such as:

- **Arithmetic Logic Units (ALUs):** Perform arithmetic and logical operations.
- **Registers:** Temporary storage locations for data being processed.
- **Buses:** Communication pathways that transfer data between components within the CPU.

An Instruction Can Be Executed By Performing One Or More Of The Following Operations

- ❖ Transfer a word of data from one processor register to another or to the ALU
- ❖ Perform an arithmetic or a logic operation and store the result in a processor register
- ❖ Fetch the contents of given memory location and load them into processor register
- ❖ Store a word of data from a processor register into a given memory location



-
- ❖ An Instruction code is a group of bits that instructs the computer to perform a specific operation.
 - ❖ It is divided into two parts and the most important parts of an instruction code is its operation code or opcode. The second part is the address part which specifies the memory address.

1. Register Transfers

- The input and output gates for register R_i are controlled by signals $R_{i_{in}}$ and $R_{i_{out}}$
 - $R_{i_{in}}$ is set to 1 – data available on common bus are loaded into R_i .
 - $R_{i_{out}}$ is set to 1 – the contents of register are placed on the bus.
 - $R_{i_{out}}$ is set to 0 – the bus can be used for transferring data from other registers .
- All operations and data transfers within the processor take place within time-periods defined by the processor clock.
- When edge-triggered flip-flops are not used, 2 or more clock-signals may be needed to guarantee proper transfer of data. This is known as multiphase clocking.

Data Transfer Between Two Registers

Example:

- Transfer the contents of R1 to R4.
- Enable output of register R1 by setting $R1_{out}=1$. This places the contents of R1 on the processor bus.
- Enable input of register R4 by setting $R4_{in}=1$. This loads the data from the processor bus into register R4.

Input and Output Gating for Registers

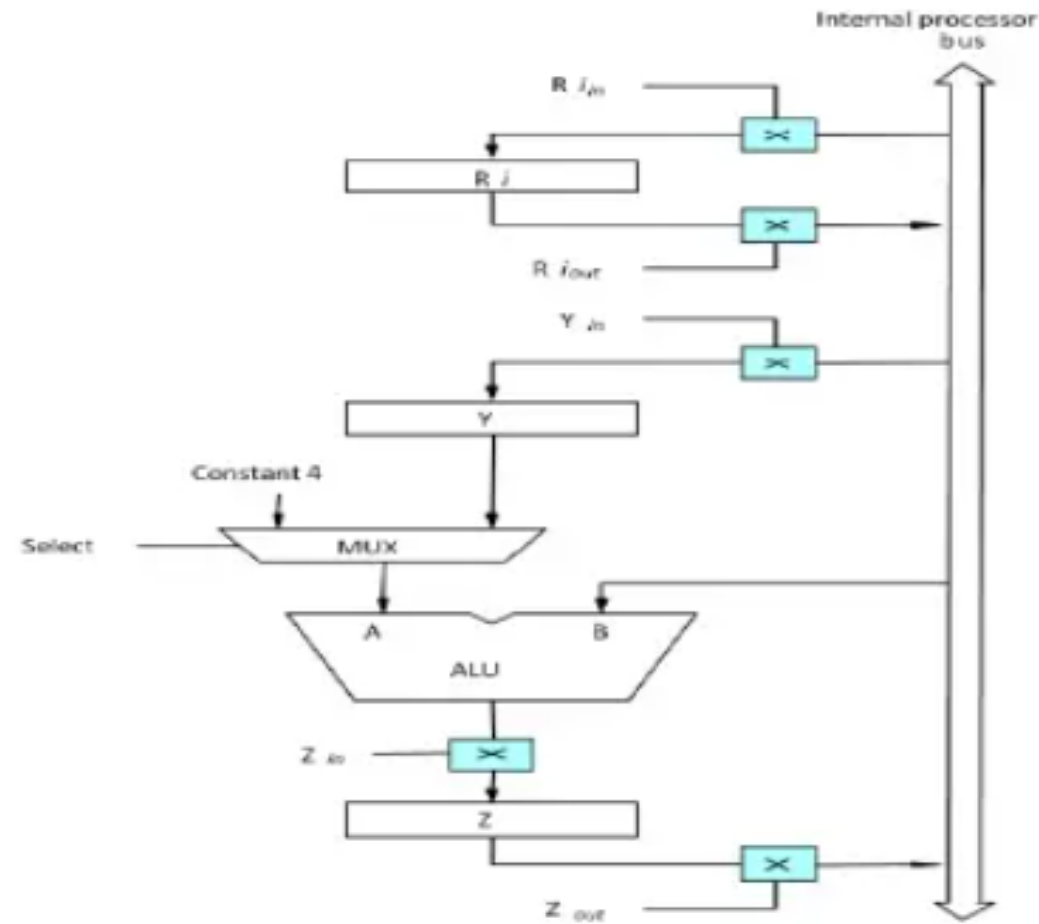


Figure. Input and Output Gating for the Registers

Register Transfers

- All operations and data transfers are controlled by the processor clock.

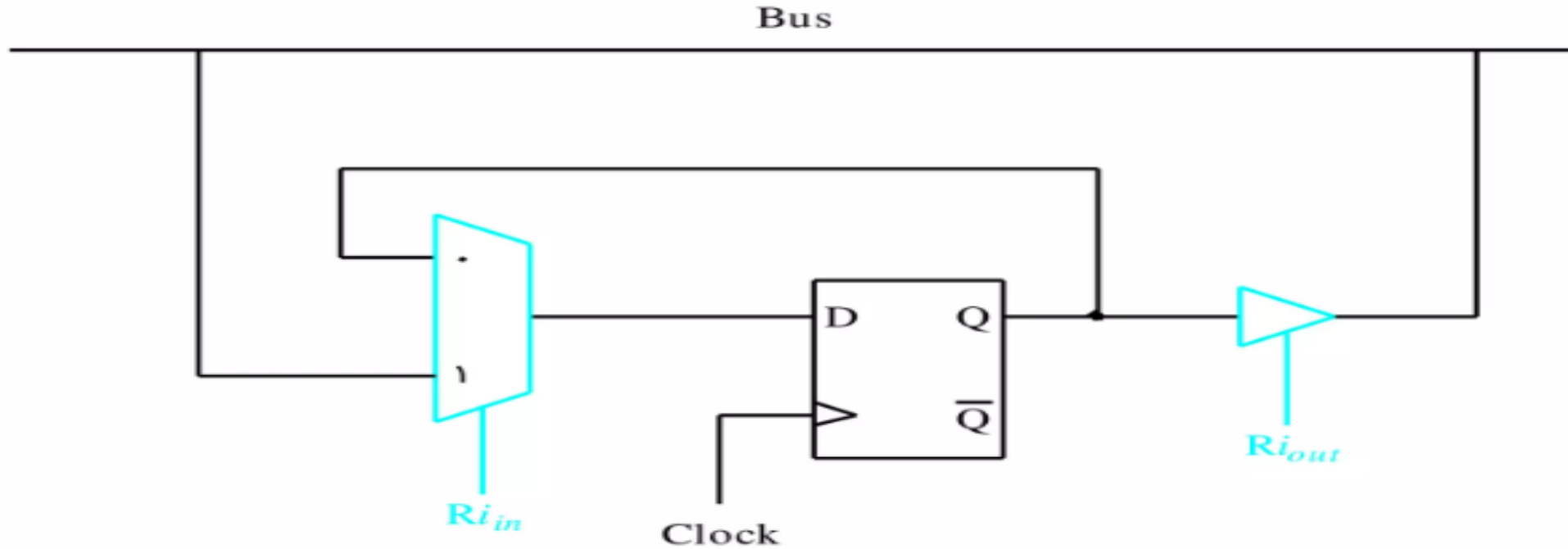


Figure 7.3. Input and output gating for one register bit.

Performing an Arithmetic or Logic Operation



- The ALU is a combinational circuit that has no internal storage.
- ALU gets the two operands from MUX and bus. The result is temporarily stored in register Z.
- What is the sequence of operations to add the contents of register R1 to those of R2 and store the result in R3?
 1. R1out, Yin
 2. R2out, SelectY, Add, Zin
 3. Zout, R3in

Fetching a Word from Memory

- Address into MAR; issue Read operation; data into MDR.

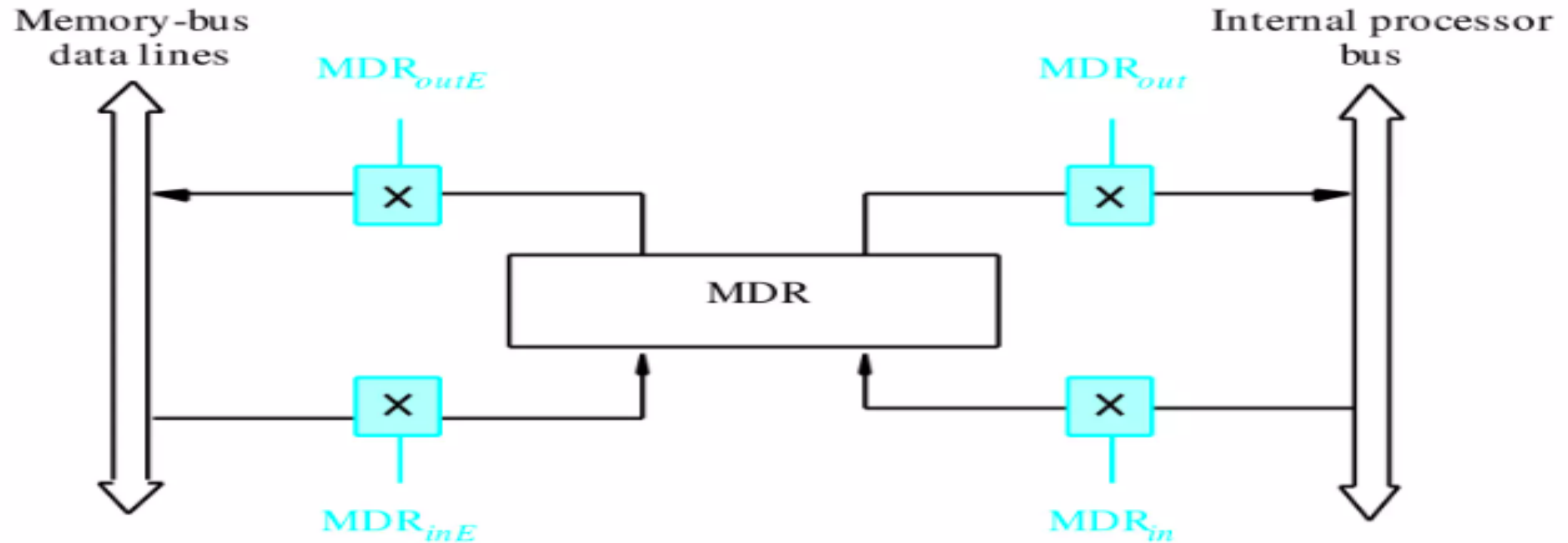


Figure 7.4. Connection and control signals for register MDR.

Storing a word in memory

- Address is loaded into MAR
- Data to be written loaded into MDR.
- Write command is issued.
- Example: Move R2,(R1)

$R1_{out}, MAR_{in}$

$R2_{out}, MDR_{in}, Write$

$MDR_{outE}, WMFC$



Fetching a Word from Memory

- The response time of each memory access varies (cache miss, memory-mapped I/O,...).
- To accommodate this, the processor waits until it receives an indication that the requested operation has been completed (Memory-Function-Completed, MFC).
- Move (R1), R2
 - $MAR \leftarrow [R1]$
 - Start a Read operation on the memory bus
 - Wait for the MFC response from the memory
 - Load MDR from the memory bus
 - $R2 \leftarrow [MDR]$

Timing

Assume MAR is always available on the address lines of the memory bus.

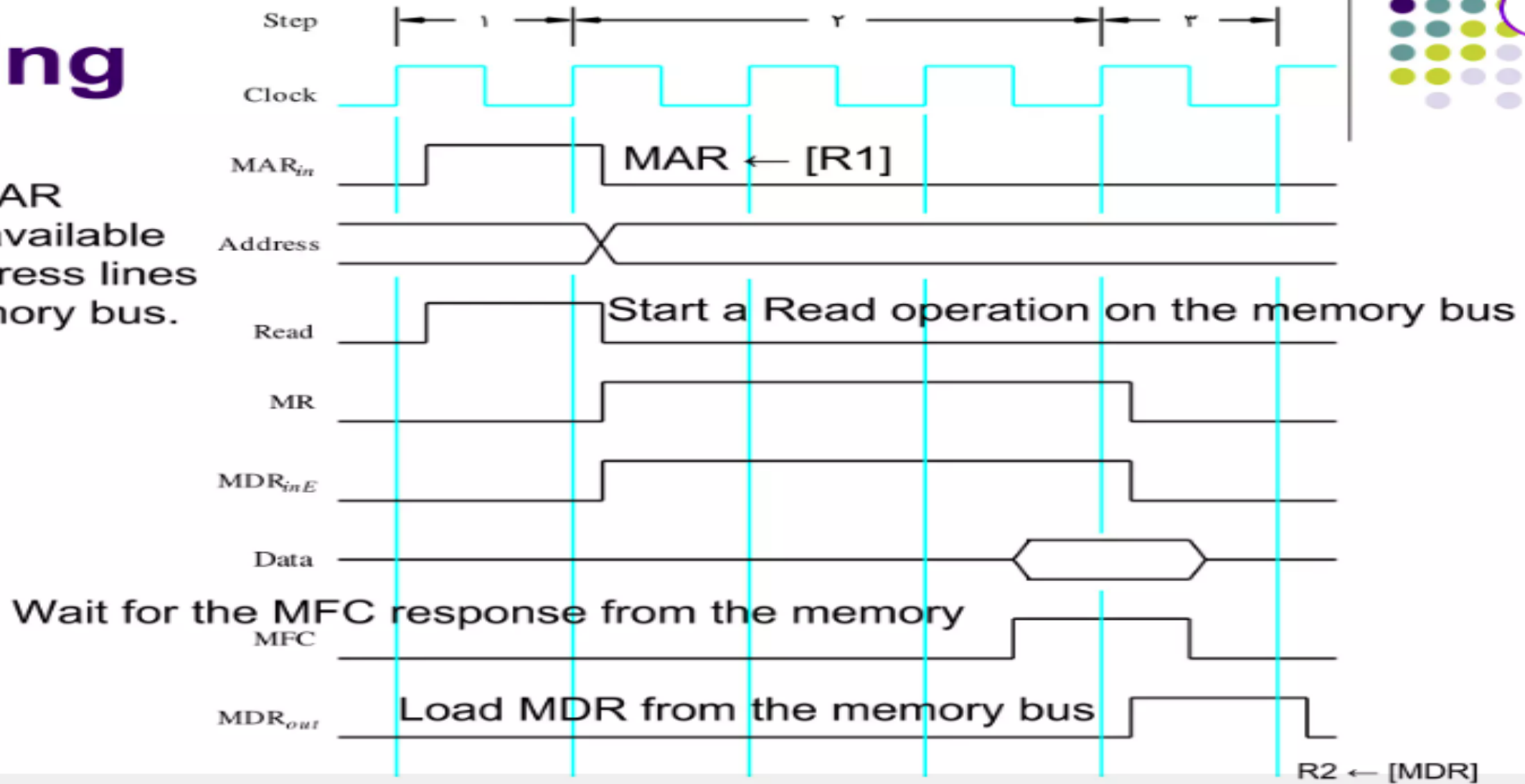
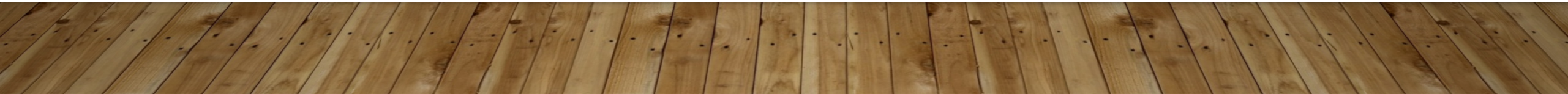


Figure V.0. Timing of a memory Read operation.

Execution of a Complete Instruction



- Add (R3), R1
- Fetch the instruction
- Fetch the first operand (the contents of the memory location pointed to by R3)
- Perform the addition
- Load the result into R1



Architecture

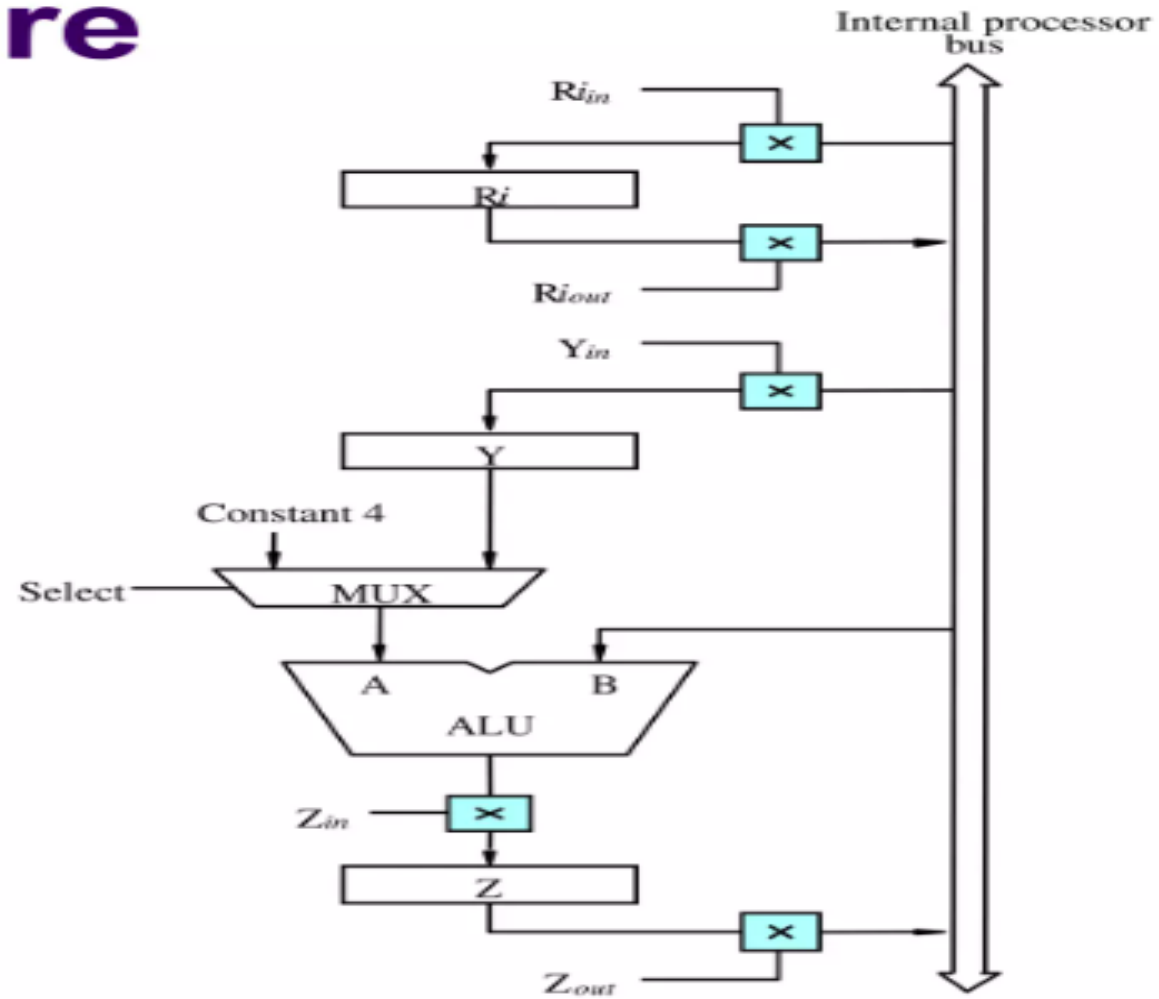


Figure 7.2. Input and output gating for the registers in Figure 7.1.

Execution of a Complete Instruction

Add (R3), R1

Step	Action
1	$PC_{out}, MAR_{in}, Read, Select \epsilon, Add, Z_{in}$
2	$Z_{out}, PC_{in}, Y_{in}, WMF C$
3	MDR_{out}, IR_{in}
4	$Rr_{out}, MAR_{in}, Read$
5	$R1_{out}, Y_{in}, WMF C$
6	$MDR_{out}, Select Y, Add, Z_{in}$
7	$Z_{out}, R1_{in}, End$

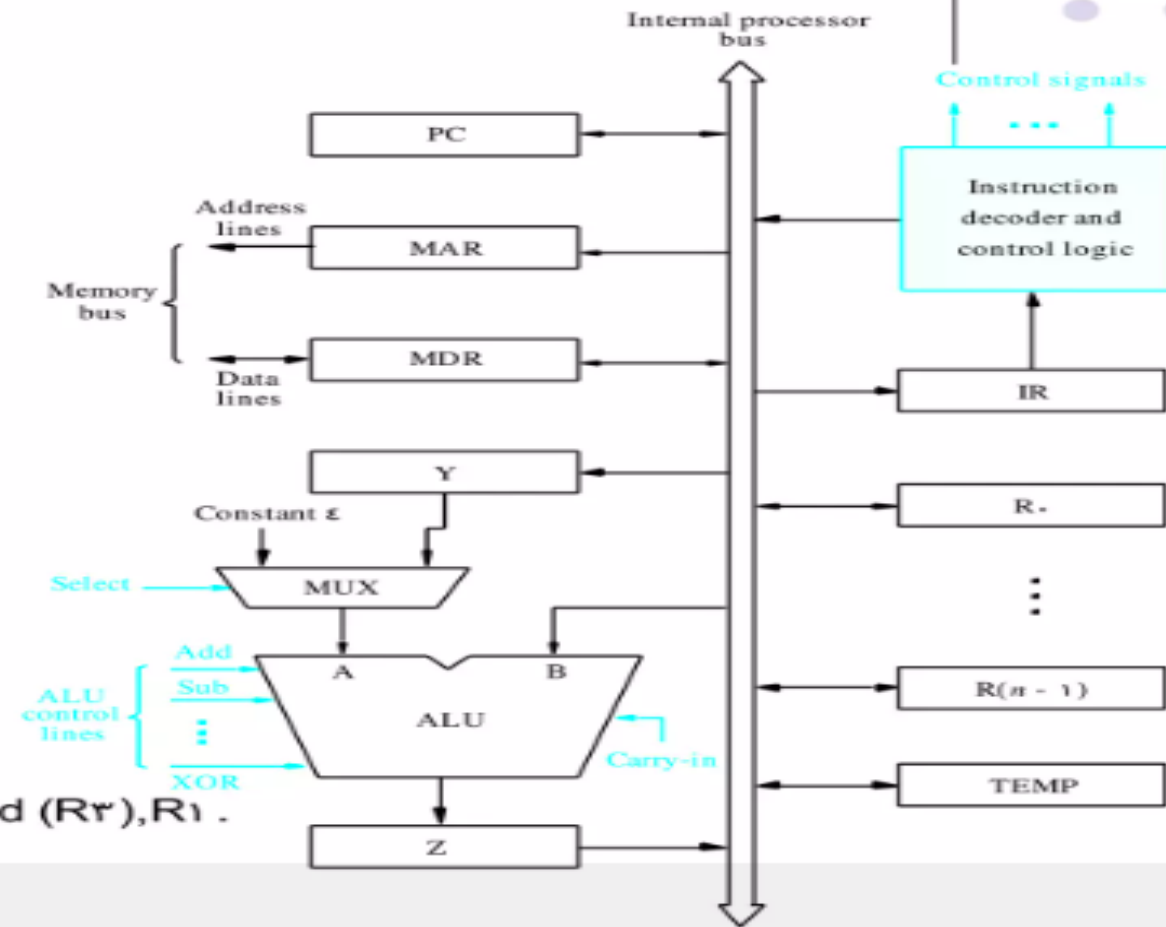


Figure V.1. Control sequence for execution of the instruction Add (Rr),R1.

Figure V.1. Single-bus organization of the datapath inside a processor.

Execution of Branch Instructions



- A branch instruction replaces the contents of PC with the branch target address, which is usually obtained by adding an offset X given in the branch instruction.
- The offset X is usually the difference between the branch target address and the address immediately following the branch instruction.
- Conditional branch

Execution of Branch Instructions



Step	Action
------	--------

- | | |
|---|--|
| 1 | PC_{out} , MAR_{in} , Read, Select4, Add, Z_{in} |
| 2 | Z_{out} , PC_{in} , Y_{in} , WMF C |
| 3 | MDR_{out} , IR_{in} |
| 4 | Offset-field-of- IR_{out} , Add, Z_{in} |
| 5 | Z_{out} , PC_{in} , End |
-

Figure 7.7. Control sequence for an unconditional branch instruction.

Quiz

- What is the control sequence for execution of the instruction

Add R1, R2

including the instruction fetch phase? (Assume single bus architecture)

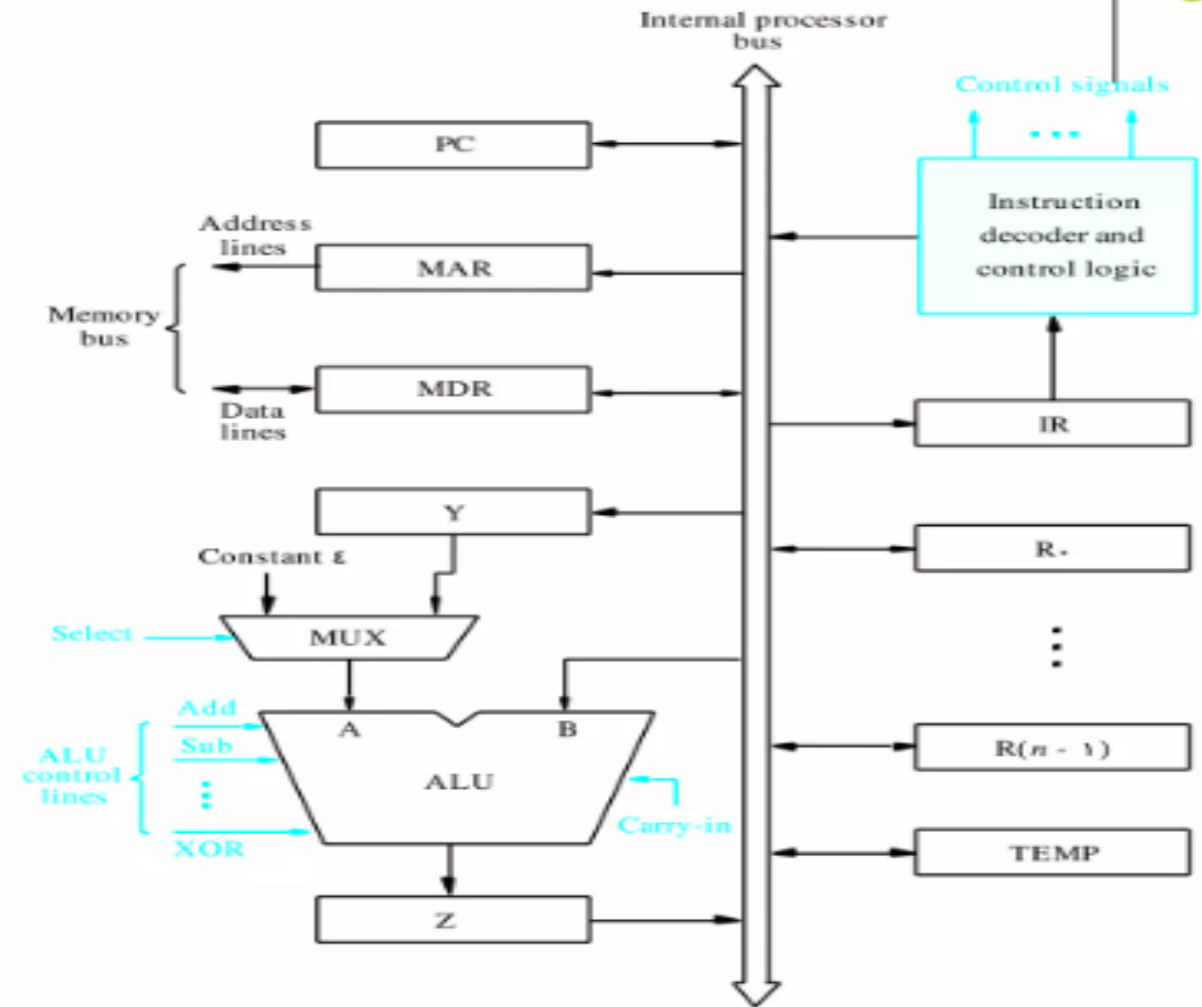


Figure V.1. Single-bus organization of the datapath inside a processor.

REGISTER TRANSFERS

- ❖ In computer architecture, register transfer refers to the process of copying data between registers, which are small, fast storage locations within the CPU.
- ❖ It involves the use of a register transfer language (RTL), a symbolic notation for describing these data movements and the micro-operations (like addition or shift) performed on the data.

-
- ❖ A register transfer statement, such as $R1 \rightarrow R2$, indicates that the contents of register $R1$ are copied into $R2$. This action requires control signals to execute and data lines to facilitate the simultaneous, parallel transfer of bits during a single clock pulse, without altering the source register.

KEY CONCEPTS

- ❖ **Registers:**

- ❖ Small, high-speed storage units within a processor used to hold data for immediate processing.

- ❖ **Micro-operations:**

- ❖ The basic operations performed on the data stored in registers, such as data transfer, addition, subtraction, or shifting.

- ❖ **Register Transfer Language (RTL):**

- ❖ A symbolic language used to describe the micro-operations and the flow of data between registers in a digital system.

- ❖ **Control Functions:**

- ❖ Control signals that initiate or allow a register transfer to occur only when a specific condition is met (e.g., $P: R1 \rightarrow R2$ only happens when control signal P is 1).

HOW REGISTER TRANSFER WORKS?

❖ **Source and Destination:**

A register transfer involves a source register (where data is read from) and a destination register (where data is written to).

❖ **Parallel Transfer:**

All bits of data from the source register are copied to the destination register simultaneously during one clock pulse.

❖ **Non-Destructive:**

The transfer is non-destructive, meaning the content of the source register remains unchanged after the data is copied.

❖ **Hardware Implementation:**

Register transfers require specific hardware logic circuits, including data lines connecting the registers and control lines to manage the transfer process.

❖ **Control:**

A control function or signal determines when the transfer takes place, allowing for conditional execution of operations.

EXAMPLE

- ❖ $R2 \leftarrow R1$: This RTL statement signifies that the data stored in register R1 is copied into register R2.
- ❖ $P: R1 \rightarrow R2$: This means that the contents of R1 are transferred to R2 only when the control signal P is active ($P = 1$).